MySQL 读写性能测试

Spring Data JPA 与 MyBatis 读写性能对比

实验日期 2026 年 10 月 18 日 | 四节点 Docker Swarm

摘要

本实验在四台 Ubuntu 18.04.6 云服务器上部署商品管理应用，比较 Spring Data JPA 与 MyBatis 在商品读取和新建场景下的响应性能。应用、MySQL、JMeter 和 Prometheus 分别运行于独立节点，通过私网完成请求发送、数据库访问和运行指标采集。两种实现共用业务接口与数据库结构，以减少业务差异对结果的影响。

测试包含两种实现、两个接口和 100/400/1000 个总线程，每个组合重复三次；各组升压 10 秒、循环 1 次。共完成 36 组、18,000 次请求、0 次失败。两种实现各 16 项真实 MySQL 功能测试全部通过；每组写入后的新增行数与成功请求数一致，读后数据量不变。

操作	实现	1000 线程吞吐量 req/s	平均响应 ms	P95 ms
读	JPA	101.7 ± 0.0	11.58	58.35
读	MyBatis	101.7 ± 0.0	8.35	36.38
写	JPA	101.0 ± 0.7	1307.02	2994.28
写	MyBatis	101.1 ± 0.1	41.33	248.80

表中为三次重复的均值，± 表示吞吐量的样本标准差，P95 列为三次 P95 的平均值。在本次 1000 线程写入条件下，MyBatis 的平均响应时间和尾延迟低于 JPA；读取场景的均值差异较小，两种实现均存在轮间波动。

1000 个线程在 10 秒内陆续启动，每个线程只发送一次请求，因此该结果描述给定请求节奏下的表现，不代表系统的最大吞吐量或持续并发容量。CPU 使用最近 5 分钟的平均变化率计算，适合观察整体负载趋势。

关键词：MySQL；Spring Data JPA；MyBatis；JMeter；Prometheus

程序地址：https://github.com/Re1lya/lab3-mysql-benchmark

1 实验目的与测试流程

本实验通过构建、部署和测试商品管理应用，掌握 Spring Boot 访问 MySQL、Docker Swarm 服务编排、JMeter 接口测试及 Prometheus 运行监控的基本方法。在相同环境中实现 JPA 与 MyBatis 两种持久层，观察它们在不同请求规模下的响应时间、吞吐量和资源使用情况。

实验程序基于 SpringAOP 项目，分别测试读取已有商品和新建商品两个业务场景。先完成环境部署与功能验证，再建立统一的数据初始状态，按预定组合执行读写测试，最后结合请求记录、数据库行数和监控时序进行分析。

项目	设置与验证方式
读取请求	CSV 提供商品 ID；GET 请求；校验 HTTP 200 与 errno=0
写入请求	随机中文及数值字段；POST /shops/1/products；校验 HTTP 201
请求执行	按参数设置总线程数，10 秒升压，每线程执行 1 次
运行监控	13 条记录规则，采集 JVM 线程数、CPU 和内存等指标
采集设置	scrape 2s；全局 evaluation 5s；规则组 30s；timeout 1s
应用资源参数	Tomcat max300/min100；Druid max300/min100/initial100
日志	业务日志级别 DEBUG；JPA show-sql=true

参数配置与可复核性

JMeter 通过 -J 参数设置 thread.num、rampup.period 和 loop.count，由测试计划中的 __P 函数读取。目标地址使用应用节点私网 IP 和 8080 端口，结果按测试组分别保存。JVM 堆大小根据节点内存设置，避免资源不足影响实验执行。

每组 JTL 的样本数应等于配置的总线程数；写入成功数还需与数据库新增行数一致。程序、测试计划、配置、原始数据和分析脚本一并保存，使用 SHA-256 校验文件完整性。数据库密码通过外部配置注入。

2 四节点环境与构建

实验使用四台云服务器，每台配置 2 核 CPU、2 GiB 内存和 40 GiB 系统盘，系统可见内存约 1800 MiB。操作系统为 Ubuntu 18.04.6，Docker 版本为 20.10.21。四台主机分别承担编译与发压、应用、数据库和监控任务。

节点	私网地址	主要任务
CDWY3	172.31.12.156	编译及 JMeter，对应服务器 A
CDWY1	172.31.1.202	应用与 Swarm Manager，对应 B
CDWY2	172.31.8.46	MySQL，对应 C
CDWY4	172.31.1.19	Prometheus，对应 D

组件	版本或参数
编译与运行	JDK 21.0.12.1；Maven 3.9.16；容器 Java 21.0.6
应用	Spring Boot 4.1.1；MyBatis 3.5.19；MyBatis-Spring 4.0.0
数据库与负载	MySQL 9.2.0；JMeter 5.6.3
监控	Prometheus 3.7.3；node-exporter 1.10.2
应用 JVM	堆 256–512 MiB；Xss 512 KiB；容器内存上限 1 GiB
JMeter JVM	堆 256–768 MiB；Xss 512 KiB；Metaspace 上限 192 MiB
报告粒度	jmeter.reportgenerator.overall_granularity=1000

构建时先安装 clonefactory，再对 core 执行 Maven install，随后通过 Maven package 打包应用，并使用 docker build 构建镜像。两种持久层使用同一应用镜像，通过 profile 切换，以保持其余程序和运行参数一致。

镜像 ID：sha256:b2e12d68ea0cd4154ab9604b48b474b84cf7b91407f558a89df875e48e9074dc。构建节点和应用节点归档校验和相同。部署入口为 4. SpringAOP/conf/docker/productdemoaop-docker.yml。

3 持久层实现与功能验证

读接口为 GET /shops/{shopId}/products/{id}，写接口为 POST /shops/{shopId}/products。两种 profile 共用 Controller、Service、DAO、DTO、数据库结构和业务事务边界。写路径新增商品后读取记录和关联信息构造响应，比较的是完整业务接口。

部分	JPA	MyBatis
仓库	Spring Data Repository	同接口的 MyBatis Repository
SQL	Hibernate 生成	Lab3Mapper.xml
事务	JPA 事务管理器	DataSourceTransactionManager
会话	持久化上下文	SqlSessionTemplate
生成字段	数据库生成创建时间回读	插入后读取数据库记录

MyBatis profile 关闭 Hibernate 与 JPA 仓库自动配置，使用 Spring 管理的 SqlSessionTemplate。[2] 两种实现均在写入后读取数据库生成的创建时间，保持返回字段语义一致。关联查询使用当前连接指定的数据库；删除测试使用独立创建、无外键关联的商品，避免影响已有数据。

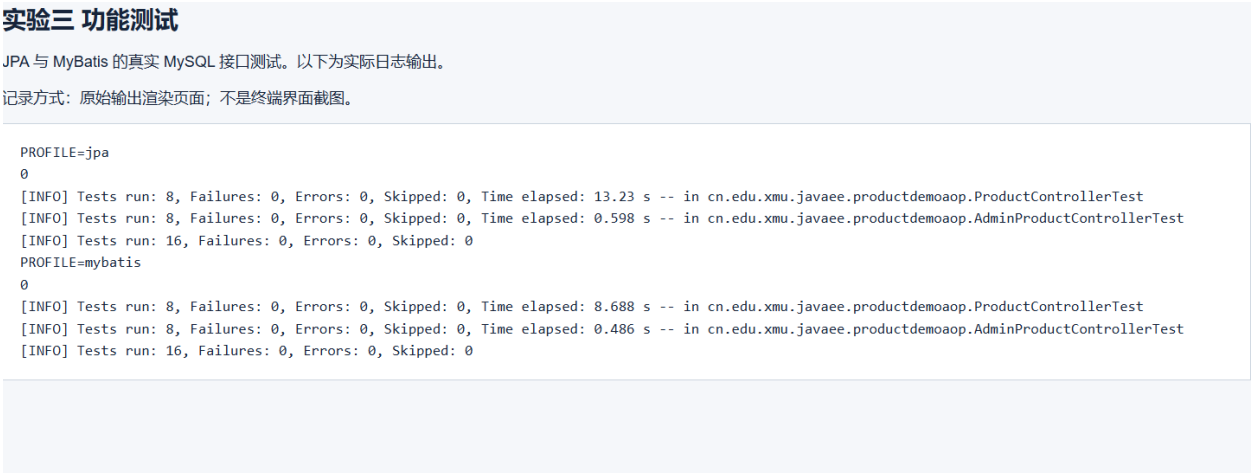


图 1 JPA、MyBatis 各 16 项功能测试通过。截图由实际测试日志渲染生成。

功能测试使用真实 MySQL 并通过事务回滚。完整日志保留在 logs/test-jpa.log 和 logs/test-mybatis.log；它们是功能验证材料，不计入性能样本。

4 数据准备与过程控制

将 product.sql 导入专用实验数据库。初始数据包含商品 3912 行、关联关系 30660 行、上架记录 3914 行，商品最大 ID 为 5461，自增起点为 5609。实验库与原运行库分开，原运行库的 3913 行商品保持不变。



图 2 数据导入后的行数和最大 ID 核验。截图由实际查询日志渲染生成。

每组前删除专用实验库中本轮新增记录，恢复初始数据和自增起点；每组后核对写入增长量与成功请求数，读后数量保持不变。所有检查语句、返回值和每组 metadata.json 均留存。

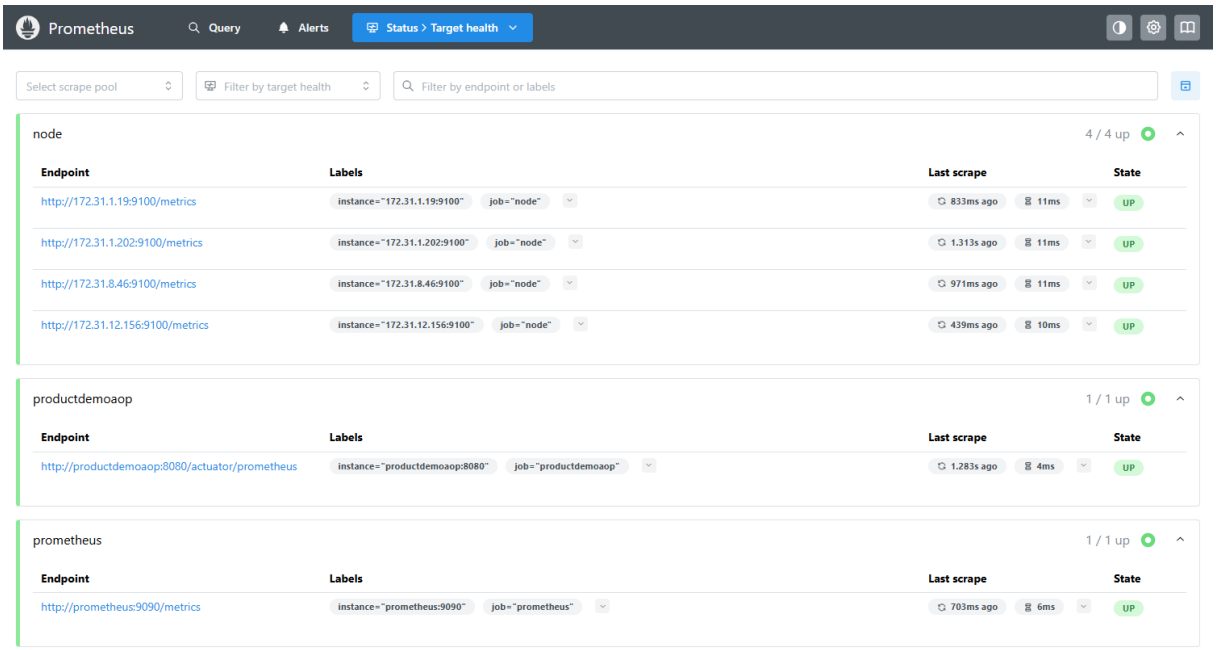


图 3 Prometheus Targets 页面，6 个监控目标均为 UP。

采集前暂停与实验无关的容器，结束后恢复原运行状态。控制程序通过心跳检测和异常处理保障恢复流程。发压期间不在负载生成节点编译或生成 HTML，结果报告与统计在本机离线生成。

5 实验设计与监控口径

因素	设置
实现	JPA、MyBatis
接口	读取、新建商品
总线程	100、400、1000
升压与循环	全部 10 秒升压、1 次循环
重复	每个组合 3 次，共 36 组、18000 次请求
次序	第 1/3 轮 JPA→MyBatis，第 2 轮 MyBatis→JPA；组内固定种子打乱
间隔	首组前等待 5 分钟；每组前空闲 15 秒，组后等待 12 秒采集监控

三个线程档位用于比较不同请求规模下的响应变化。各组采用相同的请求路径、随机数据生成方式和断言，固定升压时间与循环次数，仅改变总线程数和持久层实现。

请求统计

全程吞吐量为成功请求数除以“最后完成时间−首次发起时间”。延迟直接使用原始 JTL 的 elapsed；不扣除连接时间，不剔除升压、慢请求或异常请求。P50/P90/P95/P99 使用线性插值，重复试验以每轮为观察单位，报告均值、样本标准差与范围。JMeter HTML 采用全程样本；分位数估计器差异可能造成小幅不同。[3]

监控指标计算

CPU 使用率为 (1−空闲 CPU 时间变化率/总 CPU 时间变化率)×100%，时间变化率采用 rate[5m] 计算。内存使用率为 [1−(MemFree+Buffers+Cached)/MemTotal]×100%。JVM 活跃线程数取自 spring:jvm_threads_live_threads。

CPU 指标包含最近 5 分钟内的负载，可能受到相邻测试和部署活动影响，不能作为单组 10 秒测试的瞬时峰值。两个记录规则组每 30 秒评估一次，覆盖全局 5 秒设置；query_range 每 2 秒导出一次已记录值，重复取样不代表新的独立测量。内存统计采用 Free、Buffers 和 Cached，不与 MemAvailable 口径混用。共保存 12 类原始监控时序 JSON。[4]

6 读写性能测试结果

下表汇总三次重复；吞吐量为均值 ± 样本标准差，单位 req/s。延迟各列为每轮对应统计量的平均，单位 ms。P95 均值不等于把三轮请求合并后的整体 P95。

操作	实现	线程	吞吐量 req/s	均值 ms	P95 ms	P99 ms
读	JPA	100	10.3 ± 0.0	5.15	6.42	32.38
读	MyBatis	100	10.3 ± 0.0	4.25	4.35	28.69
读	JPA	400	40.7 ± 0.0	5.30	8.37	54.80
读	MyBatis	400	40.7 ± 0.1	3.33	4.00	36.68
读	JPA	1000	101.7 ± 0.0	11.58	58.35	183.69
读	MyBatis	1000	101.7 ± 0.0	8.35	36.38	139.68
写	JPA	100	10.2 ± 0.0	14.81	19.78	70.32
写	MyBatis	100	10.2 ± 0.0	8.95	11.35	22.85
写	JPA	400	40.5 ± 0.0	10.41	13.67	50.89
写	MyBatis	400	40.5 ± 0.1	7.66	8.37	43.03
写	JPA	1000	101.0 ± 0.7	1307.02	2994.28	3318.22
写	MyBatis	1000	101.1 ± 0.1	41.33	248.80	569.08

累计 18,000 次请求、0 次失败。写测试的数据库增量与成功请求数逐组一致；读测试没有改变商品行数。所有逐次统计见 analysis/all-runs.csv。

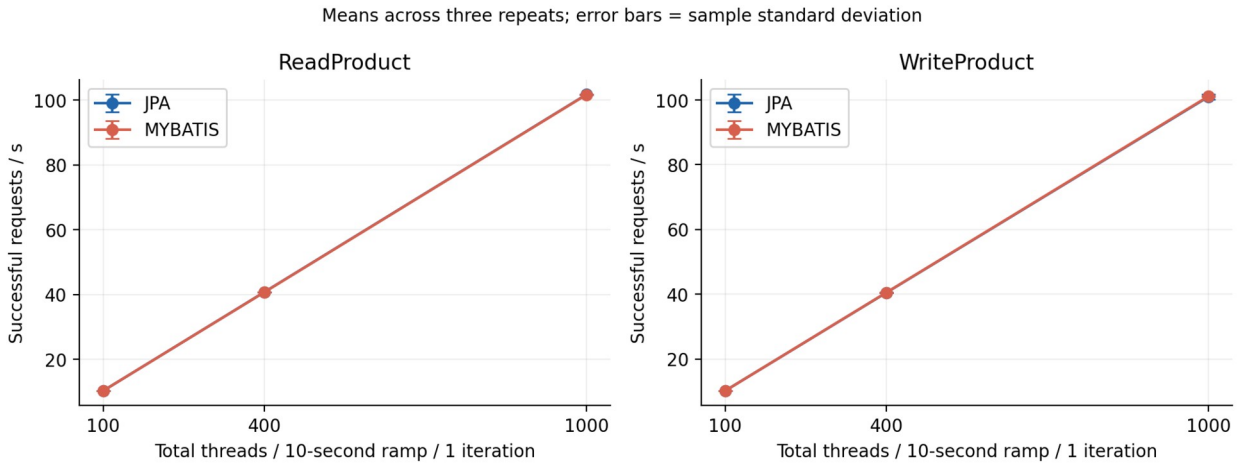


图 4 各线程档位的成功吞吐量。请求节奏受 10 秒升压约束，结果不代表饱和容量。

7 延迟与重复波动

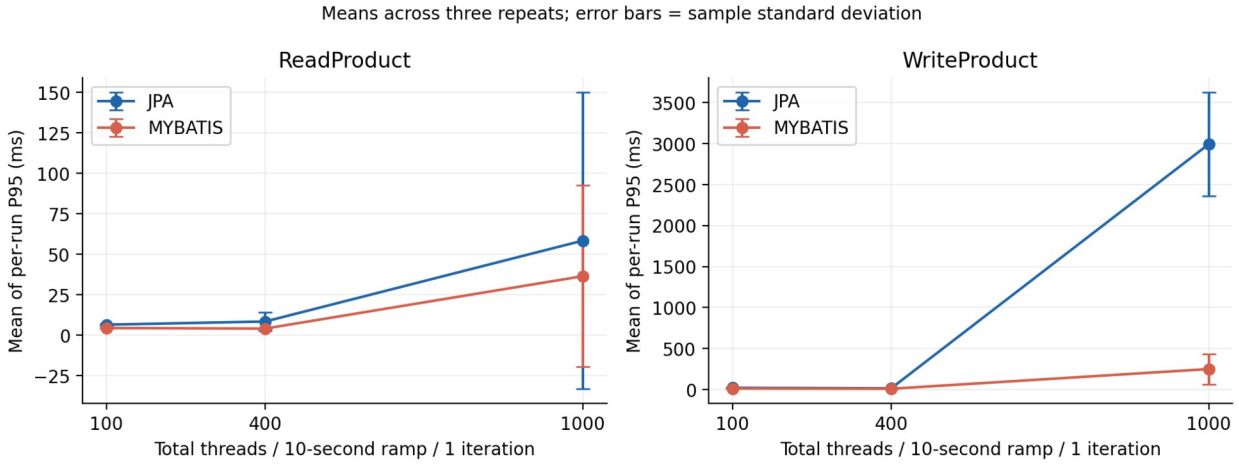


图 5 各组 P95 的三轮均值和样本标准差。

操作与实现	线程	第 1 轮均值	第 2 轮均值	第 3 轮均值	标准差
读 JPA	100	5.14	5.39	4.91	0.24
读 MyBatis	100	4.01	4.76	3.99	0.44
读 JPA	400	4.17	3.69	8.05	2.39
读 MyBatis	400	3.03	3.03	3.94	0.52
读 JPA	1000	3.05	26.69	5.01	13.12
读 MyBatis	1000	3.66	18.04	3.36	8.39
写 JPA	100	17.84	12.26	14.32	2.82
写 MyBatis	100	9.85	8.16	8.85	0.85
写 JPA	400	11.21	9.32	10.68	0.98
写 MyBatis	400	8.24	7.64	7.10	0.57
写 JPA	1000	1383.62	549.26	1988.18	722.51
写 MyBatis	1000	51.55	16.45	55.98	21.66

表中延迟单位为 ms。每轮切换持久层实现时重新部署应用，未进行长时间预热；冷启动、JIT 编译和缓存积累可能影响早期请求。三轮结果均纳入统计，以均值和样本标准差描述响应水平及其波动。

8 响应分布与时间变化

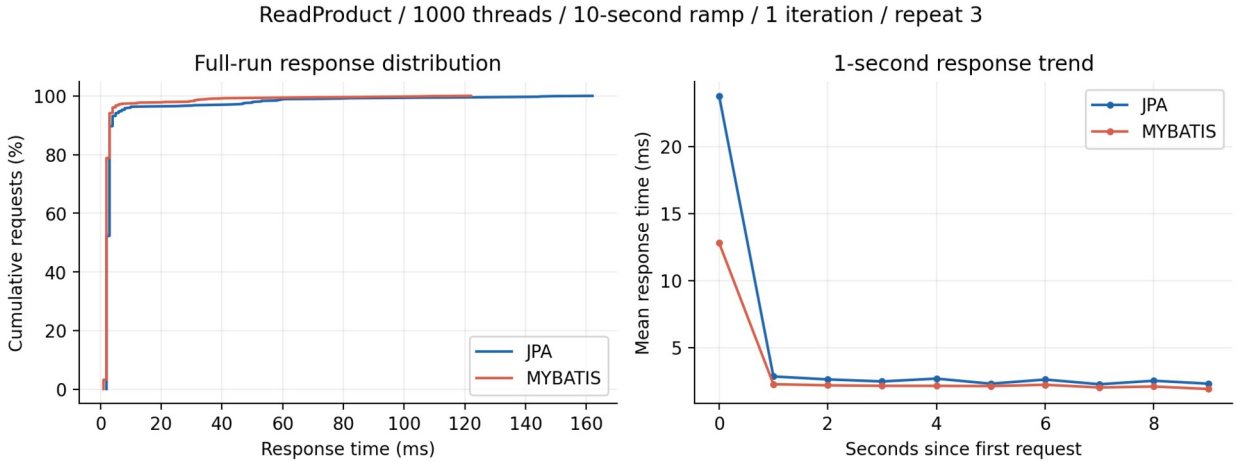


图 6 第 3 轮 1000 线程读取的完整延迟分布与每秒平均响应耗时。

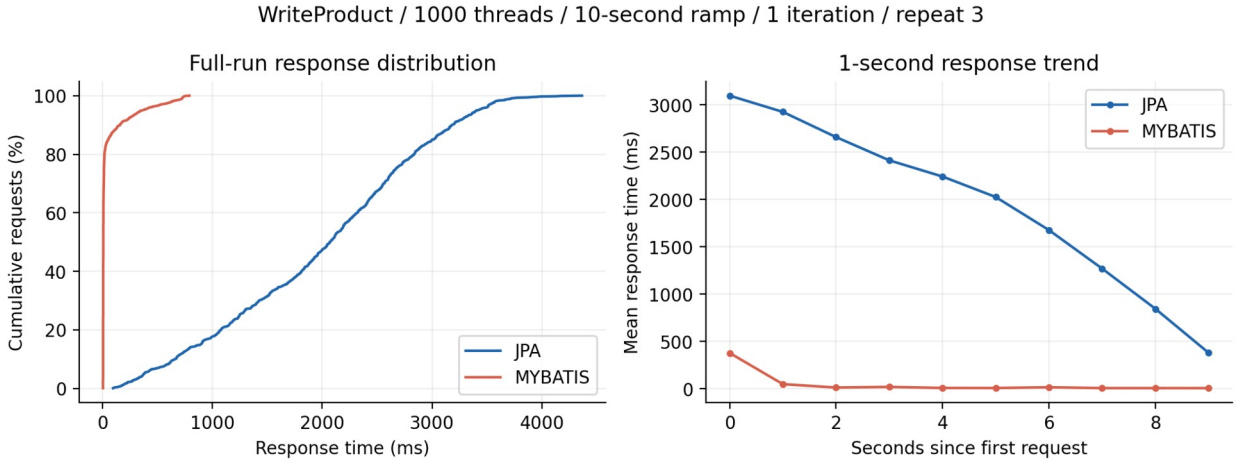


图 7 第 3 轮 1000 线程写入的完整延迟分布与每秒平均响应耗时。

以上选取第 3 轮 1000 线程的读写测试，展示完整响应分布及每秒平均耗时。线程在 10 秒内陆续启动，完成一次请求后退出，因此配置的总线程数与同时活跃的线程数不同。

9 运行指标的时间变化

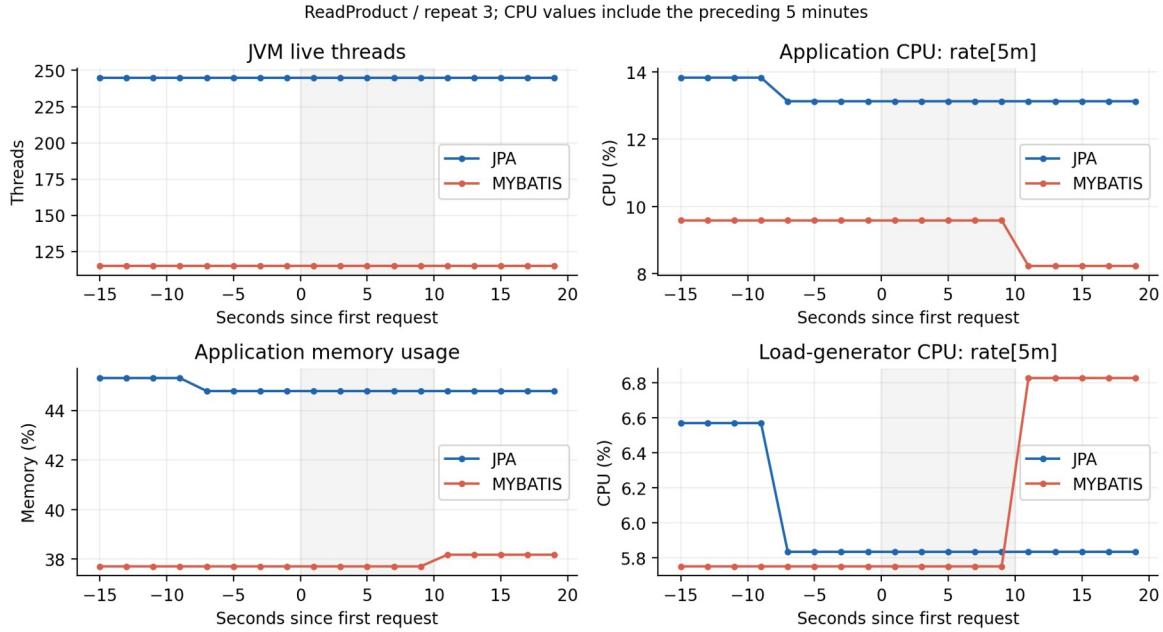


图 8 读取过程的 JVM 线程数、应用 CPU、应用内存及发压 CPU；CPU 采用 5 分钟窗口。

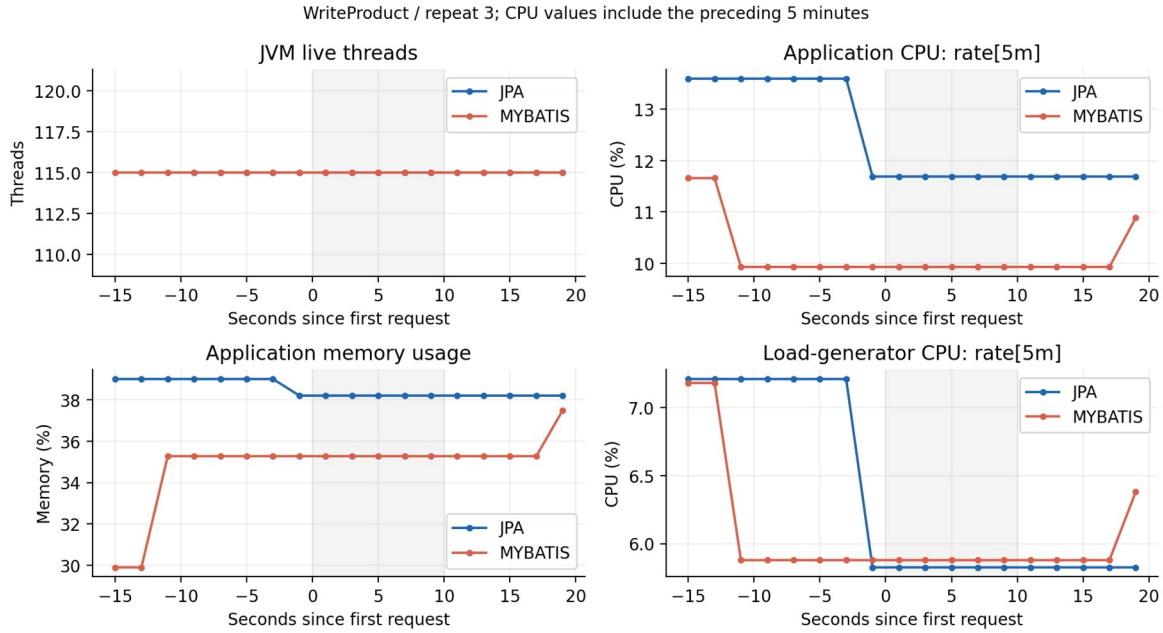


图 9 写入示例同类指标。横轴各自对齐首个请求，两种实现不是同时发压。

10 JMeter 原始报告与截图

以下为根据原始 JTL 生成的 JMeter HTML 报告截图，汇总粒度为 1000 ms。36 份完整报告分别保存，样本数、失败数、平均响应时间、极值和吞吐量均与独立统计结果核对一致。

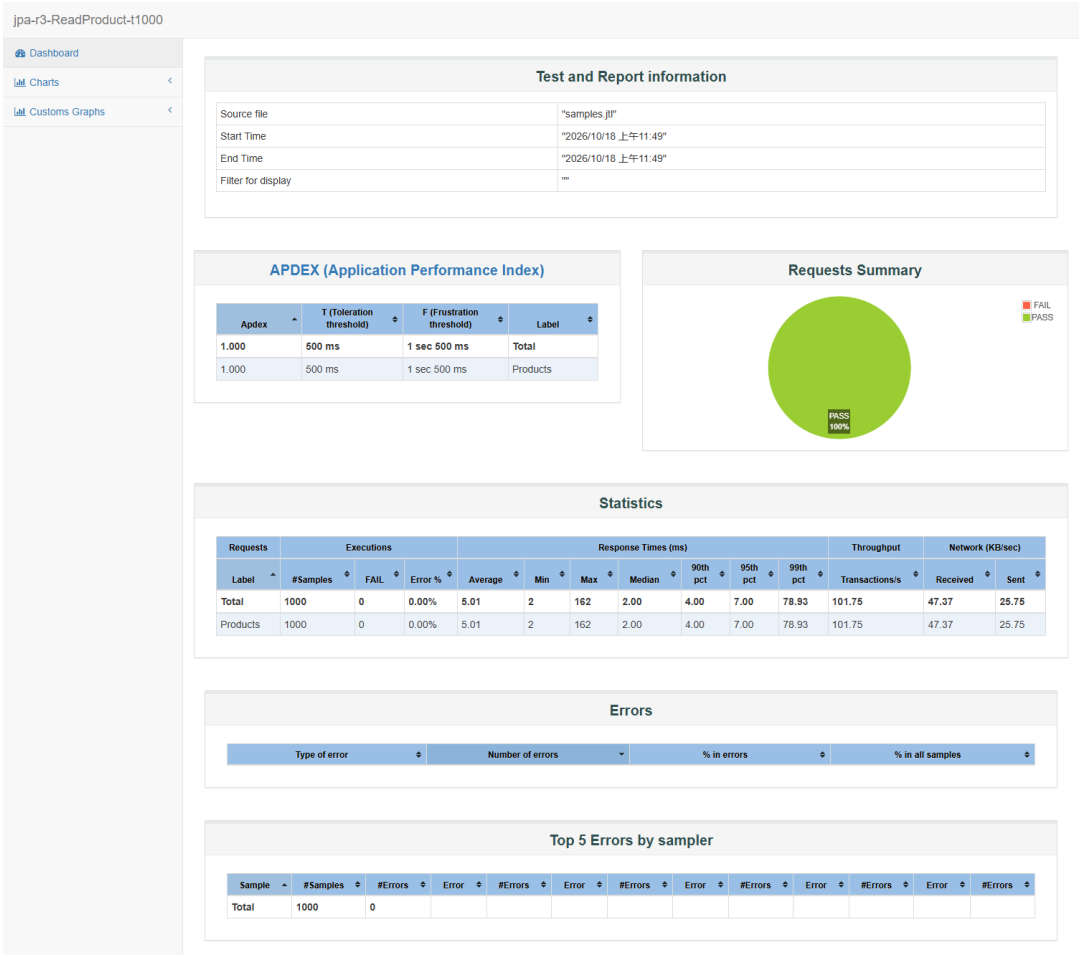


图 10 JPA 第 3 轮 1000 线程读取，JMeter 实际 HTML 报告页面。完整分辨率原图另附。

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	**1000**	**0**	**0.00%**	**55.98**	**6**	**792**	**9.00**	**160.90**	**372.55**	**726.00**	**101.04**	**47.83**	**37.67**
HTTP Request	1000	0	0.00%	55.98	6	792	9.00	160.90	372.55	726.00	101.04	47.83	37.67

图 11 MyBatis 第 3 轮 1000 线程写入，JMeter 实际统计区域截图。

1000 线程、10 秒升压、1 次循环的各组均生成 1000 条样本。附加截图包括 JPA 写入、MyBatis 读取及 JVM 线程、CPU、内存查询页面。JMeter 截图来自采集后生成的 HTML，功能验证截图由实际日志渲染生成。

11 结果解释与适用范围

在 1000 线程、10 秒升压、1 次循环的读取条件下，JPA 平均响应为 11.58 ms，MyBatis 为 8.35 ms；对应 P95 均值为 58.35 ms 与 36.38 ms。三次重复的样本标准差已在表中给出，均值差异仅描述本次运行。

在 1000 线程、10 秒升压、1 次循环的写入条件下，JPA 平均响应为 1307.02 ms，MyBatis 为 41.33 ms；对应 P95 均值为 2994.28 ms 与 248.80 ms。三次重复的样本标准差已在表中给出，均值差异仅描述本次运行。

两种实现的吞吐量均接近总请求数除以升压时间，说明本次测试的请求节奏对吞吐量有较强约束。相近的吞吐量并不表示极限处理能力相同，应结合平均响应时间、尾延迟和实际活动线程分析给定负载下的表现。

JVM 线程包含工作线程和维护线程，不等于 JMeter 并发。CPU 的 5 分钟窗口覆盖邻近测试和部署活动，适合观察整体趋势；不能凭该曲线单独推断某条 SQL、锁竞争或磁盘是唯一瓶颈。本实验没有采集火焰图或锁等待证据。

结果适用于本次 2 核/2 GiB 云主机、3912 条初始商品和所测业务路径。持续并发、混合读写、长期运行及更大数据量下的表现仍需进一步测试。零失败表示本次请求断言和数据库行数核验均通过，不能代替对其他业务场景的验证。

恢复与留存

测试结束后，应用恢复为 JPA 模式并连接原运行库，无关容器恢复原运行状态。服务副本、应用健康和 6 个监控目标检查正常，专用实验库恢复初始数据。原始 JTL 以 gzip 无损压缩保存，并附 SHA-256；统计 CSV、HTML 和图表均可通过原始 JTL 与监控 JSON 复核。

源码、配置、镜像和数据库备份分别保留，以便回滚与复现实验。程序和实验材料统一保存在公开代码仓库，登录凭据不纳入提交材料。

12 实验完成情况与资料

页码	指导书要求	本次材料
1	SpringBoot、MySQL、RESTful 读写	两种 profile、相同接口、真实 MySQL 功能测试
1	四台 Ubuntu 主机分工	第 2 节的节点配置与任务分配
2	JMeter5.6.3、PATH、内存参数	环境日志与截图、运行脚本
2	1000ms 报告粒度、目标 IP 端口	user.properties、JMX 与生成日志
3	clonefactory、core、应用与镜像	构建日志和镜像校验和
3	私网监控、targets/rules	6 个 UP 目标、13 条记录规则及页面截图
3	线程、CPU 和内存查询	指标计算方法、时序 JSON 与监控截图
3	1000/10/1 测试与 JTL/HTML	1000 线程 12 组；100/400 线程 24 组
4	Git 地址、原始数据、报告	个人公开仓库、原始数据包、本报告
4	程序与原始数据验收	程序、原始数据和报告已整理，待提交审核

Git 地址：https://github.com/Re1lya/lab3-mysql-benchmark

随报告提供原始 JTL、监控时序、36 份 HTML 报告、逐次与分组统计、过程截图及完整性校验清单。分析脚本与程序代码一同保存，可按测试组定位和复核结果。

参考资料

[1] 《实验三：MySQL 读写性能测试》，第 1–4 页；课程源码 https://gitee.com/mingqcn/spring-fundamentals，提交 fce17e7。

[2] MyBatis-Spring，Using an SqlSession。https://mybatis.org/spring/sqlsession.html

[3] Apache JMeter，Generating Dashboard Report。https://jmeter.apache.org/usermanual/generating-dashboard.html

[4] Prometheus，HTTP API。https://prometheus.io/docs/prometheus/latest/querying/api/

在线文档核对日期：2026 年 10 月 18 日。